

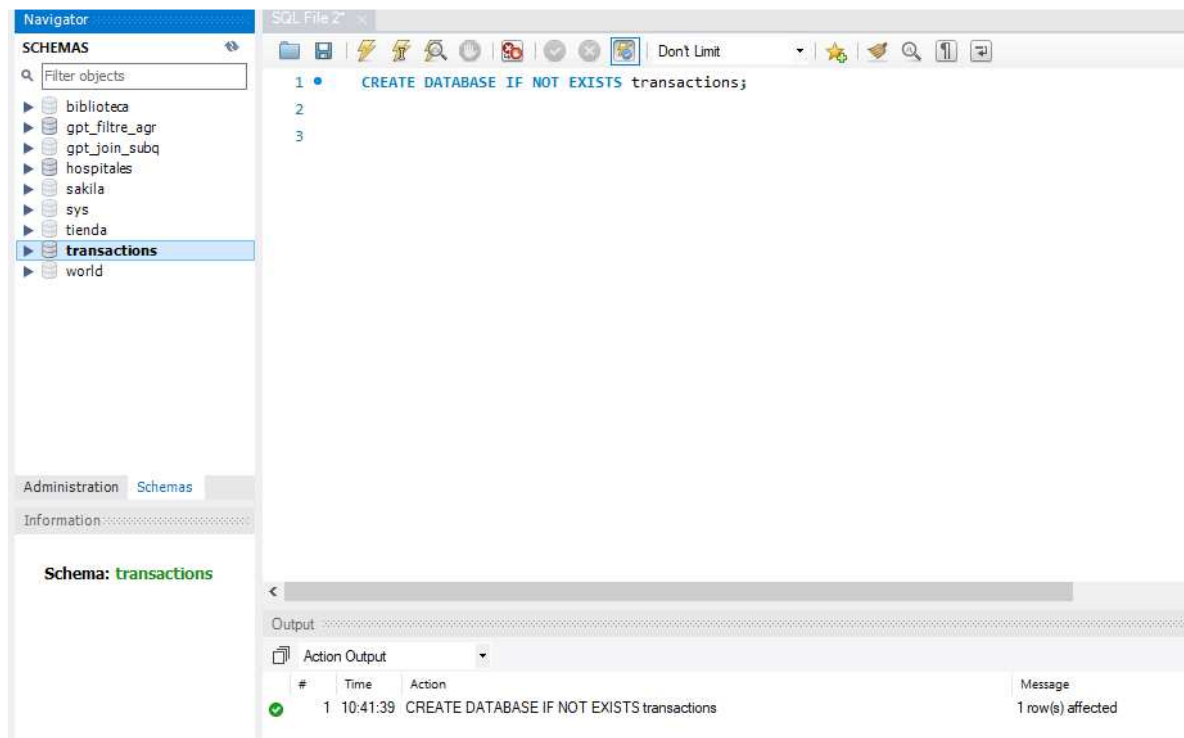
Nivel 1

Ejercicio 1

A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

Se crea una BBDD y las tablas solicitadas a partir de los documentos SQL facilitados, haciendo correr los códigos.

Creación de la BBDD Transactions:



Se usa el código CREATE DATABASE IF NOT EXISTS, que señala la creación de una BBDD en el caso de que no exista ya. De existir una BBDD con el mismo nombre daría error y no crearía la BBDD.

Creación de las tablas company y transaction:

The screenshot shows the SQL Enterprise Manager interface. On the left, the 'SCHEMAS' pane shows the 'transactions' schema selected. The main pane displays the SQL script for creating two tables:

```
4 CREATE TABLE IF NOT EXISTS company (  
5     id VARCHAR(15) PRIMARY KEY,  
6     company_name VARCHAR(255),  
7     phone VARCHAR(15),  
8     email VARCHAR(100),  
9     country VARCHAR(100),  
10    website VARCHAR(255)  
11 );  
12 CREATE TABLE IF NOT EXISTS transaction (  
13     id VARCHAR(255) PRIMARY KEY,  
14     credit_card_id VARCHAR(15) REFERENCES credit_card(id),  
15     company_id VARCHAR(20),  
16     user_id INT REFERENCES user(id),  
17     lat FLOAT,  
18     longitude FLOAT,  
19     timestamp TIMESTAMP,  
20     amount DECIMAL(10, 2),  
21     declined BOOLEAN,  
22     FOREIGN KEY (company_id) REFERENCES company(id)  
23 );
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message
3	10:43:38	USE transactions	0 row(s) affected
4	10:43:48	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, com...	0 row(s) affected
5	10:43:55	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, cr...	0 row(s) affected

Seguidamente, se añade la información facilitada para rellenar las tablas previamente creadas:

The screenshot shows the SQL Enterprise Manager interface with the 'transactions' schema selected. The main pane displays the SQL script for inserting data into the 'company' and 'transaction' tables:

```
1 USE transactions;  
2  
3 -- Insertamos datos de company  
4 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-222'  
5 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-222'  
6 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-223'  
7 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-223'  
8 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-223'  
9 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-224'  
10 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-224'  
11 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-225'  
12 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-225'  
13 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-225'  
14 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-226'  
15 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-226'  
16 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-227'  
17 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-227'  
18 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-227'  
19 INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-228'
```

The 'Output' pane at the bottom shows the execution results for the 'transaction' table:

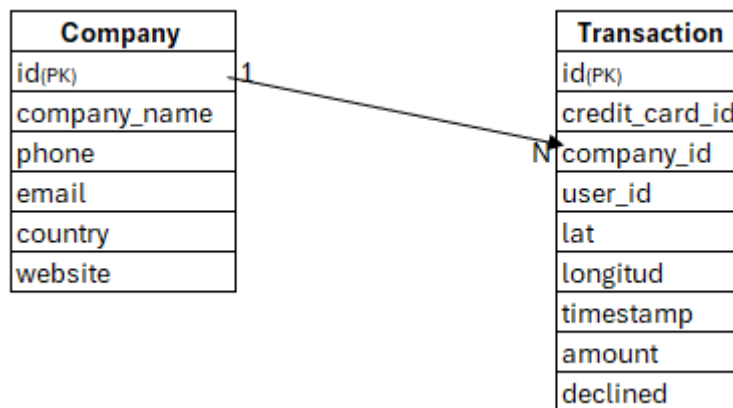
#	Time	Action	Message
100105	16:00:06	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, a...	1 row(s) affected
100106	16:00:06	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, a...	1 row(s) affected
100107	16:00:07	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, a...	1 row(s) affected

Se observa que la tabla **transaction** es una tabla de hechos donde se muestra cada transacción generada, con su id correspondiente y establecido como Primary Key. A este lo acompaña información relevante: tarjeta, compañía,

usuario, localidad con la latitud y longitud, fecha y hora y cantidad de la transacción en cuestión junto si fue aprobada o no.

Tanto la referencia de la tarjeta, compañía y usuario son mediante de sus id, pudiendo extraer la información detallada de estos en sus respectivas tablas, siendo estas tablas de dimensiones. Mostrando un modelo relacional en estrella.

En esta parte del ejercicio solo se dispone de la tabla de dimensiones **company**, donde se muestra la información asociada a cada una de las compañías que realizan las transacciones reflejadas en la tabla de hechos. Cada uno de los registros de la tabla, pertenece a una compañía determinada, con su propia clave de referencia, siendo esta la Primary Key de la tabla utilizándose como campo para unirse a la tabla transaction , en una relación de uno a muchos.

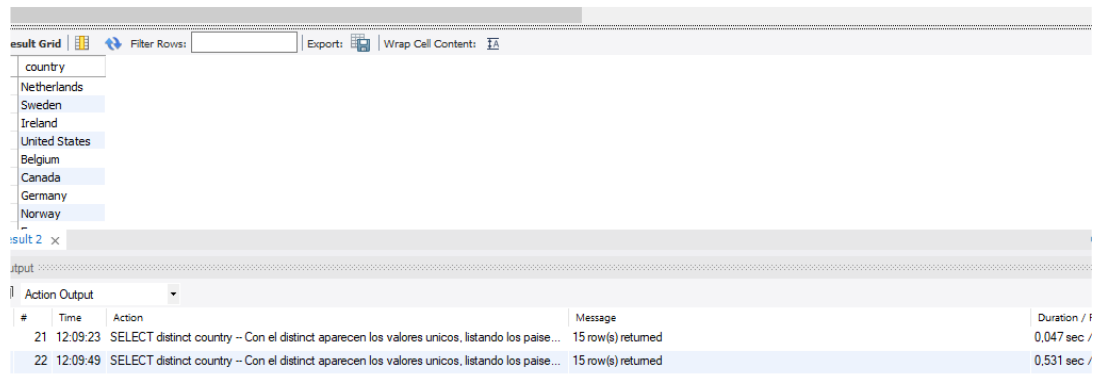


Ejercicio 2

Utilitzant JOIN realitzaràs les següents consultes:

- *Llistat dels països que estan generant vendes.*

```
8 # Ejercicio 2. Utilizando JOINS:
9 # 2.1 Llistat dels països que estan generant vendes.
10 • SELECT distinct country -- Con el distinct aparecen los valores unicos, listando los paises una unica vez y no por tantas como aparezcan.
11 FROM company
12 JOIN transaction -- Se realiza la JOIN para ver que empresas han realizado transacciones. Quedando fuera las que no han realizado ninguna
13 ON company.id = transaction.company_id
14 WHERE declined = 0;
```



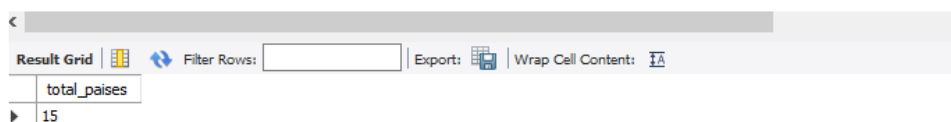
#	Time	Action	Message	Duration / f
21	12:09:23	SELECT distinct country -- Con el distinct aparecen los valores unicos, listando los paise...	15 row(s) returned	0,047 sec /
22	12:09:49	SELECT distinct country -- Con el distinct aparecen los valores unicos, listando los paise...	15 row(s) returned	0,531 sec /

Se realiza un INNER JOIN para ver que compañías han realizado operaciones.

Se filtra por las transacciones que NO han sido declinadas.

- *Des de quants països es generen les vendes.*

```
16 # 2.2 Des de quants països es generen les vendes.
17 • SELECT count(DISTINCT country) AS total_paises -- Utilizo el count distinct para que solc
18 FROM company
19 JOIN transaction
```



total_paises
15



#	Time	Action	Message
---	------	--------	---------

Con la misma INNER JOIN, se calcula con un COUNT(), el numero de compañías que han realizado operaciones y no que el pago no ha sido declinado.

- *Identifica la companyia amb la mitjana més gran de vendes.*

```

23 # 2.3 Identifica la companyia amb la mitjana més gran de vendes.
24 • SELECT company_name, avg(amount) AS media_ventas
25 FROM company
26 JOIN transaction
27 ON company.id = transaction.company_id
28 WHERE declined = 0
29 GROUP BY company_name
30 ORDER BY media_ventas DESC
31 LIMIT 1;

```

Result Grid

company_name	media_ventas
Ac Fermentum Incorporated	284.911333

Result 5 x

Output

Action Output

#	Time	Action	Message
1	12:14:13	SELECT company_name, avg(amount) AS media_ventas FROM company JOIN transacti...	1 row(s) returned

Misma INNER JOIN pero ahora se solicita función de agregación con la media de las ventas con pago aceptado en cada compañía.

Una INNER JOIN es lo mismo que una JOIN, muestra los valores coincides en ambas tablas según valores en condición.

Ejercicio 3

Utilitzant només subconsultes (sense utilitzar JOIN).

- Mostra totes les transaccions realitzades per empreses d'Alemanya.

```

# Ejercicio 3. Utilizando Subqueries:
# 3.1 Mostra totes les transaccions realitzades per empreses d'Alemanya.
• SELECT *
FROM transaction
WHERE company_id IN (SELECT id
FROM company
WHERE country = "Germany");

```

Result Grid

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0

Action 9 x

Output

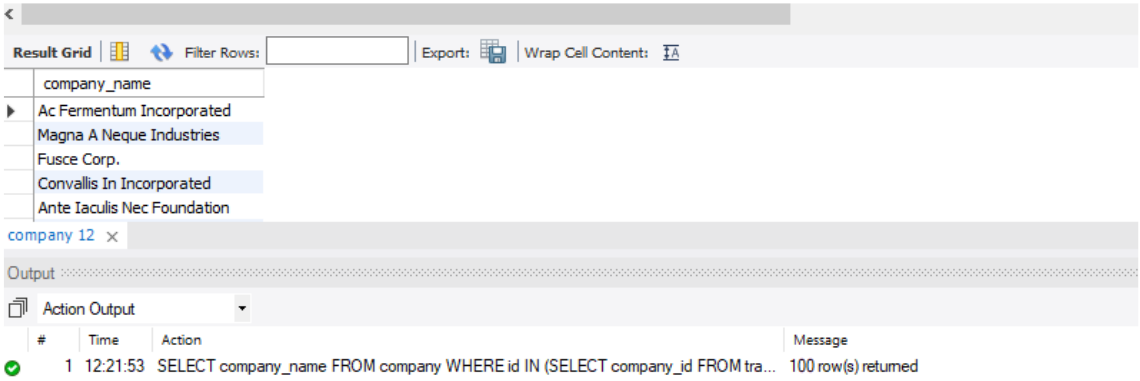
Action Output

#	Time	Action	Message
1	12:17:26	SELECT * FROM transaction WHERE company_id IN (SELECT id FROM company	13291 row(s) returned

Se muestran todas las operaciones, tanto con pagos aceptados como no, realizadas por empresas de Alemania.

- *Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.*

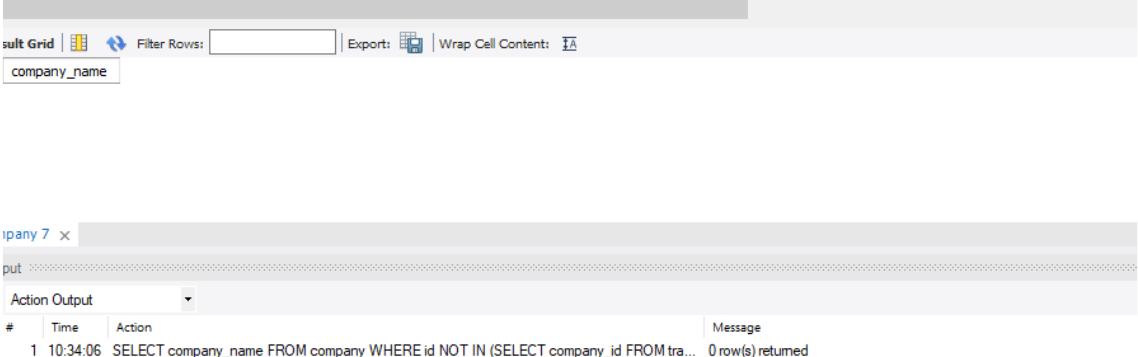
```
41 # 3.2 Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes le
42 • SELECT company_name
43 FROM company
44 WHERE id IN (SELECT company_id
45             FROM transaction
46             WHERE amount > (SELECT AVG(amount)
47                             FROM transaction));
48
```



La lista de las empresas que han realizado transacciones, tanto aceptadas como no, por una media superior a la media global.

- *Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.*

```
7 # 3. Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.
8 • SELECT company_name
9 FROM company
10 WHERE id NOT IN (SELECT company_id
11                 FROM transaction);
```



No hay empresas que no tengan transacciones registradas, en el caso de que hubiera, el código para eliminar las compañías sin transacciones sería el siguiente:

```
52
53 #Si se quisiera borrar las compañías (filas) que no presentan transacciones en la tabla transacciones.
54 • DELETE FROM company
55 WHERE id NOT IN (SELECT DISTINCT company_id
56 FROM transaction);
```

Output

#	Time	Action	Message	Duration / Fetch
2	10:44:41	DELETE FROM company WHERE id NOT IN (SELECT company_id FROM transaction)	Error Code: 1175. You are using safe update mode and you tried to update a table without a ...	0.094 sec

Como esta activado el safe mode se ha de desactivar, ejecutar la eliminación de filas y volver a activar el safe mode:

```
>4
53 #Si se quisiera borrar las compañías (filas) que no presentan transacciones en la tabla transacciones.
54 • SET SQL_SAFE_UPDATES = 0;
55 • DELETE FROM company
56 WHERE id NOT IN (SELECT DISTINCT company_id
57 FROM transaction);
58 • SET SQL_SAFE_UPDATES = 1;
```

Output

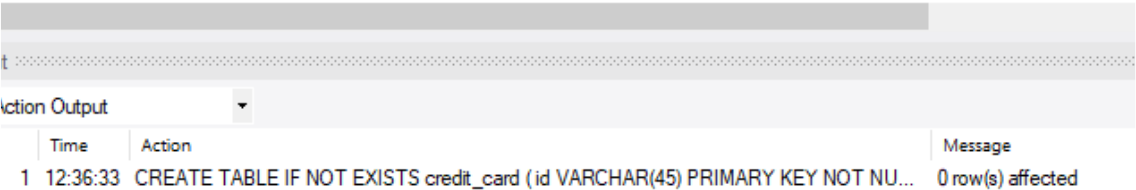
#	Time	Action	Message	Duration / Fetch
8	11:19:22	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
9	11:19:22	DELETE FROM company WHERE id NOT IN (SELECT DISTINCT company_id FROM transa...	0 row(s) affected	0.219 sec
10	11:19:22	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec

Ejercicio 4

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

Primeramente, se crea la estructura de la tabla con los datos facilitados en "dades_introduir_credit":

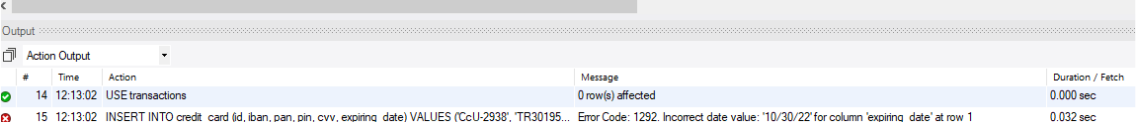
```
#Ejercicio 4. Creación nueva tabla en la BBDD:
-- Creacion tabla:
CREATE TABLE IF NOT EXISTS credit_card (
    id VARCHAR(45) PRIMARY KEY NOT NULL,
    iban VARCHAR(50),
    pan VARCHAR(50),
    pin VARCHAR (50),
    cvv VARCHAR (10),
    expiring_date DATE);
```



The screenshot shows a database management tool interface. At the top, there's a text area with SQL code for creating a table. Below it, an 'Action Output' window displays the execution results. The output shows a single line: '1 12:36:33 CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(45) PRIMARY KEY NOT NU... 0 row(s) affected'. The table has columns: Time, Action, and Message.

Seguidamente cargamos los datos facilitados en "dades_introduir_credit" en una hoja nueva de SQL.

```
8 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2973', 'PT87806228135092429456346', '544 58654 54343 384', '8768', '887', '01
9 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2980', 'DE39241881883086277136', '402400 7145845969', '5075', '596', '07/24/2
10 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2987', 'GE89681434837748781813', '3763 747687 76666', '2298', '797', '10/31/2
11 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2994', 'BH62714428368066765294', '344283273252593', '7545', '595', '02/28/22'
12 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3001', 'CY49087426654774581266832110', '511722 924833 2244', '9562', '867', '
13 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3008', 'LU507216693616119230', '4485744464433884', '1856', '740', '04/05/25')
14 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3015', 'PS119398216295715968342456821', '3784 662233 17389', '3246', '822', '
15 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3022', 'GT91695162850556977423121857', '5164 1379 4842 3951', '5610', '342', '
16 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3029', 'AZ62317413982441418123739746', '3429 279566 77631', '9708', '505', '0
17 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3036', 'AZ39336002925842865843941994', '3768 451556 48766', '2232', '565', '1
18 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3043', 'TN6488143310514852179535', '455676 6437463635', '5969', '196', '06/07
19 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3050', 'FR5167744369175836831854477', '4024007123722', '4834', '126', '10/09/
```



The screenshot shows a database management tool interface. It displays a list of SQL insert statements. Below the statements, an 'Output' window shows the execution results. The output table has columns: #, Time, Action, Message, and Duration / Fetch. The first row shows a successful execution of 'USE transactions'. The second row shows an error: '15 12:13:02 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR30195... Error Code: 1292. Incorrect date value: '10/30/22' for column 'expiring_date' at row 1'.

Da error al no coincidir el formato de la fecha en los datos facilitados con el tipo de dato DATE. Para hacer que coincida, primero se modifica el tipo de dato del campo expiring_date a VARCHAR, para poder subir la información a la tabla.

Después se modifica el formato de los datos en la estructura requerida y se cambiara a tipo de dato DATE.

1. Cambio de tipo de dato en campo expiring_date:

```
67 cvv varchar(10),
68 expiring_date DATE);
69 -- el tipo de dato en expiring_date no es correcto, modificamos tipo de dato para que coincida con la fecha facilitada:
70 -- para ello, primero modifiko el tipo de dato que es el campo expiring_date VARCHAR, para poder subir la informacion a la tabla.
71 ALTER TABLE credit_card
72 MODIFY COLUMN expiring_date VARCHAR(10);
73
74
75
```

Table: credit_card

Columns:

- id: varchar(45) PK
- iban: varchar(50)
- pan: varchar(50)
- pin: varchar(50)
- cvv: varchar(10)
- expiring_date: varchar(10)

Object Info Session

Output

#	Time	Action	Message	Duration / F
14	12:13:02	USE transactions	0 row(s) affected	0.000 sec
15	12:13:02	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', ...	Error Code: 1292. Incorrect date value: '10/30/22' for column 'expiring_date' at row 1	0.032 sec
16	12:40:51	ALTER TABLE credit_card MODIFY COLUMN expiring_date VARCHAR(10)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	2.437 sec

2. Se cargan los datos en la tabla:

```
11 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2994', '8H62714428368066765294', '344283273252593', '7545', '595', '02/28/2
12 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3001', 'CV49887426654774581266832110', '511722 924833 2244', '9562', '867',
13 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3008', 'LU507216693616119230', '4485744464433884', '1856', '740', '04/05/25
14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3015', 'PS119398216295715968342456821', '3784 662233 17389', '3246', '822',
15 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3022', 'GT91695162850556977423121857', '5164 1379 4842 3951', '5610', '342',
16 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3029', 'AZ62317413982441418123739746', '3429 279566 77631', '9788', '505',
17 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3036', 'AZ39336002925842865843941994', '3768 451556 48766', '2232', '565',
18 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3043', 'TN6488143310514852179535', '455676 6437463635', '5969', '196', '06/
19 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3050', 'FR5167744369175836831854477', '4024007123722', '4834', '126', '10/0
```

Output

#	Time	Action	Message	Duration / F
5015	13:00:24	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9579', 'XX296393...	1 row(s) affected	0.500 sec
5016	13:00:25	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX781258...	1 row(s) affected	0.188 sec
5017	13:00:25	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX915670...	1 row(s) affected	0.375 sec

Datos cargados a la tabla:

```
72 -- comprobacion datos en tabla:
73
74 SELECT *
75 FROM credit_card;
76
```

Result Grid

id	iban	pan	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	2314242385113924	1819	467	09/27/25
CcS-4858	XX8581768137002436094025	6582720299715533	3964	817	12/28/28
CcS-4859	XX7826930491423553609370	8861684536289642	4983	277	11/26/26
CcS-4860	XX5559590368835304645299	2481155515498459	6876	661	07/27/27
CcS-4861	XX2035182877195191627307	1308930301149557	5710	398	04/25/26
CcS-4862	XX4774721462463645409758	6715617009807829	4042	174	11/27/26

credit_card 9 x

Output

#	Time	Action	Message
5017	13:00:25	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX91567...	1 row(s) affected
5018	13:08:17	SELECT * FROM credit_company	Error Code: 1146. Table 'transactions.credit_company' doesn't exist
5019	13:08:28	SELECT * FROM credit_card	5000 row(s) returned

3. Se modifica el formato de la fecha a la que acepta el tipo de dato DATE:

Para poder actualizar los datos hemos de desactivar el Safe update mode, realizar la actualización y por último reactivar el safe update mode.

```
-- modificamos el formato de la fecha facilitada: (desactivamos y reactivamos el safe update mode)
SET SQL_SAFE_UPDATES = 0;
UPDATE credit_card
SET expiring_date = STR_TO_DATE(expiring_date, "%m/%d/%y");
SET SQL_SAFE_UPDATES = 1;
```

Time	Action	Message
21 13:18:54	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
22 13:18:56	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, "%m/%d/%y")	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0
23 13:19:05	SET SQL_SAFE_UPDATES = 1	0 row(s) affected

4. Se vuelve a cambiar el tipo de dato del campo expiring_date a tipo DATE:

Functions

world

Administration Schemas

Information

Table: credit_card

Columns:

id	varchar(45)
iban	PK
pan	varchar(50)
pin	varchar(50)
cvv	varchar(10)
expiring_date	date

Object Info Session

```
81 -- Comprobamos cambio:
82 • SELECT *
83 FROM credit_card;
84 -- Pasamos el tipo de dato del campo expiring_date de varchar a date
85 • ALTER TABLE credit_card
86 MODIFY COLUMN expiring_date DATE;
87
```

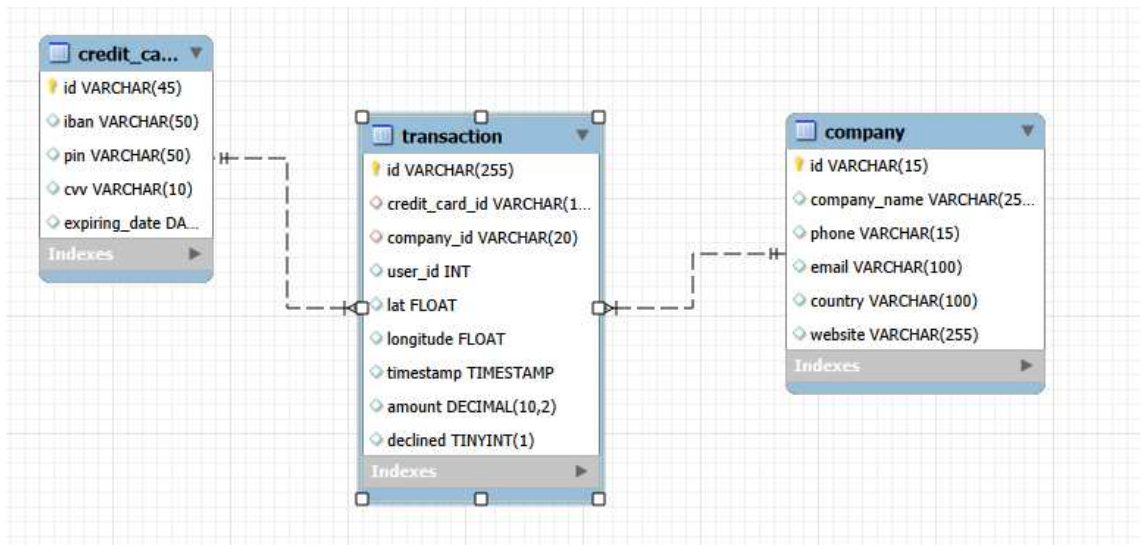
#	Time	Action	Message
5024	13:19:22	SELECT * FROM credit_card	5000 row(s) returned
5025	13:19:40	SELECT * FROM credit_card	5000 row(s) returned
5026	13:21:13	ALTER TABLE credit_card MODIFY COLUMN expiring_date DATE	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Así mismo se crea la restricción Foreign Key con la tabla de hechos transactions para facilitar la integridad de los datos.

```
91 -- Se establece restricción Foreign Key
92 • ALTER TABLE transaction
93 ADD CONSTRAINT trans_ccfk_2 FOREIGN KEY (credit_card_id) REFERENCES credit_card(id);
94
```

Visible	Key	Type	Uni...	Columns
☒	PRIMARY	BTREE	YES	id
☒	transaction_ibfk_1	BTREE	NO	company_id
☒	trans_ccfk_2	BTREE	NO	credit_card_id

El modelo en estrella de la BBDD transactions presenta el siguiente diagrama:



Se ha creado una nueva tabla de dimensiones, que se relaciona con la tabla de hechos via el campo id en credit_card, que es la primary key de la tabla, a credit_card_id en la tabla transaction como Foreign Key con una relación de uno a muchos.

Ejercicio 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

Se visualiza la tarjeta con el ID CcU-2938 para confirmar error:

```
89 #Ejercicio 5
90 -- visualizamos tarjeta para ver si error
91 • SELECT id, iban
92 FROM credit_card
93 WHERE id = "CcU-2938";
94
```

result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

id	iban
CcU-2938	TR301950312213576817638661
NULL	NULL

edit_card 15 x

Output

Action Output

#	Time	Action	Message
5029	13:31:33	SELECT * FROM transaction	100000 row(s) returned
5030	13:39:13	SELECT * FROM credit_card	5000 row(s) returned
5031	13:41:23	SELECT id, iban FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned

Se realiza cambio, previamente se desactiva el safe updatemode y posteriormente se reactiva.

```
94 -- actualizamos la informacion (necesito desactivar safe update mode, realizamos cambio y lo volvemos a reactivar):
95 • SET SQL_SAFE_UPDATES = 0;
96 • UPDATE credit_card
97 SET iban = "TR323456312213576817699999"
98 WHERE id = "CcU-2938";
99 • SET SQL_SAFE_UPDATES = 1;
100
101
```

Output

Action Output

#	Time	Action	Message
✓ 1	13:52:38	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
✓ 2	13:52:42	UPDATE credit_card SET iban = "TR323456312213576817699999" WHERE id = "CcU-2938"	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0
✓ 3	13:52:48	SET SQL_SAFE_UPDATES = 1	0 row(s) affected

Comprobación cambio:

100 -- comprobacion cambio

101 • SELECT id, iban

102 FROM credit_card

103 WHERE id = "CcU-2938";

104

<

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	id	iban
▶	CcU-2938	TR323456312213576817699999
*	NULL	NULL

credit_card 16 ×

Output

Action Output

#	Time	Action	Message
✓ 2	13:52:42	UPDATE credit_card SET iban = "TR323456312213576817699999" WHERE id = "CcU-2...	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0
✓ 3	13:52:48	SET SQL_SAFE_UPDATES = 1	0 row(s) affected
✓ 4	13:54:05	SELECT id, iban FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned

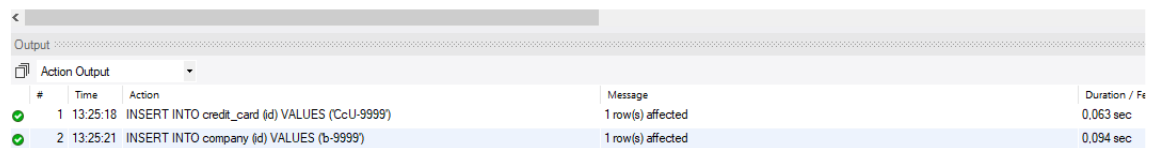
Ejercicio 6

En la taula "transaction" ingresa una nova transacció amb la següent informació:

Para ello se inserta una nueva fila con comando INSERT INTO.

Antes de subir el nuevo registro en transaction se han de subir los nuevos registros credit_card_id y company_id a sus respectivas tablas, para que las tablas padre (de dimensión) contemplen los nuevos registros a insertar en la tabla hija (tabla de hechos transaction) y así mantener la integridad de los datos.

```
115 -- Da error en la restricción FK, al querer subir registros en los campos credit_card_id y company_id que no se encuentran en las tablas padre:
116 -- Añadir registro credit_card_id CcU-9999 en la tabla credit_card:
117 • INSERT INTO credit_card (id) VALUES ('CcU-9999');
118 -- Añadir registro business_id b-9999 en la tabla company:
119 • INSERT INTO company (id) VALUES ('b-9999');
```



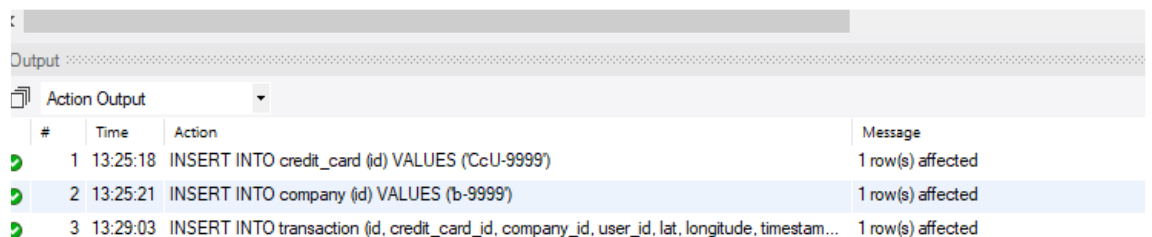
#	Time	Action	Message	Duration / Fetch
1	13:25:18	INSERT INTO credit_card (id) VALUES ('CcU-9999')	1 row(s) affected	0.063 sec
2	13:25:21	INSERT INTO company (id) VALUES ('b-9999')	1 row(s) affected	0.094 sec

Al subir el registro, este da error, ya que el campo timestamp no permite valores nulos y no se ha facilitado valor para este campo en el enunciado. Para poder subir la información se establece la fecha a día de la subida con la función NOW().

```
104
105 #Ejercicio 6
106 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('10881D1D-5B23-A76C-55EF-C568E4
```


#	Time	Action	Message	Duration / Fetch
1	17:15:37	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, a...	Error Code: 1292. Incorrect datetime value: "for column 'timestamp' at row 1	0.062 sec

```
119 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amou
120
```



#	Time	Action	Message
1	13:25:18	INSERT INTO credit_card (id) VALUES ('CcU-9999')	1 row(s) affected
2	13:25:21	INSERT INTO company (id) VALUES ('b-9999')	1 row(s) affected
3	13:29:03	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestam...	1 row(s) affected

Ejercicio 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recordar mostrar el canvi realitzat.

Para ello primero se comprueba que aparece dicha columna:

```
123 SELECT *
124 FROM credit_card;
```

The screenshot shows a database interface with a 'Result Grid' displaying the columns of the 'credit_card' table: id, iban, pan, pin, cvv, and expiring_date. Below the grid, the 'Output' section shows the 'Action Output' for the executed query, indicating that 5000 rows were returned.

#	Time	Action	Message
14	18:30:09	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestam...	1 row(s) affected
15	18:32:52	SELECT * FROM credit_card	5000 row(s) returned

La columna existe y se ve en la información de la tabla que no presenta ninguna restricción.

Se elimina columna:

```
125 -- se procede a eliminar columna
126 ALTER TABLE credit_card
127 DROP COLUMN pan;
```

The screenshot shows the 'Output' section with the 'Action Output' for the executed commands. The first command (ALTER TABLE) shows that 5000 rows were returned. The second command (DROP COLUMN) shows that 0 rows were affected, with 0 duplicates and 0 warnings.

#	Time	Action	Message
15	18:32:52	SELECT * FROM credit_card	5000 row(s) returned
16	18:35:55	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Se comprueba que eliminación de la columna:

```
128 -- Se comprueba que no existe campo:
129 SELECT *
130 FROM credit_card;
```

The screenshot shows the 'Result Grid' displaying the columns of the 'credit_card' table: id, iban, pin, cvv, and expiring_date. The 'pan' column is no longer present. Below the grid, the 'Output' section shows the 'Action Output' for the executed query, indicating that 5001 rows were returned.

#	Time	Action	Message
2	13:25:21	INSERT INTO company (id) VALUES (b-9999)	1 row(s) affected
3	13:29:03	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestam...	1 row(s) affected
4	13:32:32	SELECT * FROM credit_card	5001 row(s) returned
5	13:32:36	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Record
6	13:32:39	SELECT * FROM credit_card	5001 row(s) returned

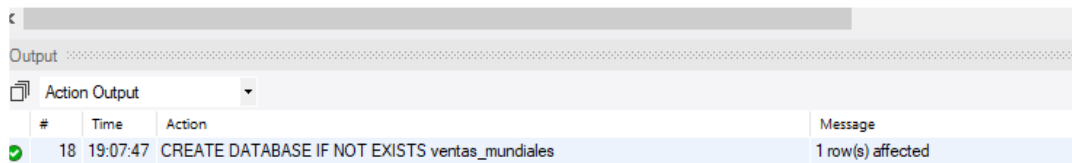
Ejercicio 8

Descarrega els arxius CSV que **trobaràs a l'apartat de recursos**: *american_users.csv*, *european_users.csv*, *companies.csv*, *credit_cards.csv*, *transactions.csv*.

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

Creación de la BBDD y se ejecuta comando USE seleccionar trabajar con ella.

```
132 #Ejercicio 8
133 -- Se crea BBDD nueva:
134 • CREATE DATABASE IF NOT EXISTS ventas_mundiales;
135 • USE ventas_mundiales;
```

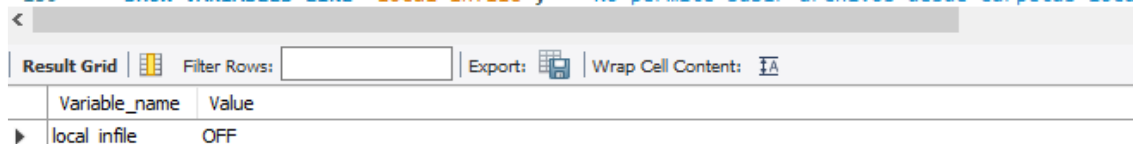


#	Time	Action	Message
18	19:07:47	CREATE DATABASE IF NOT EXISTS ventas_mundiales	1 row(s) affected

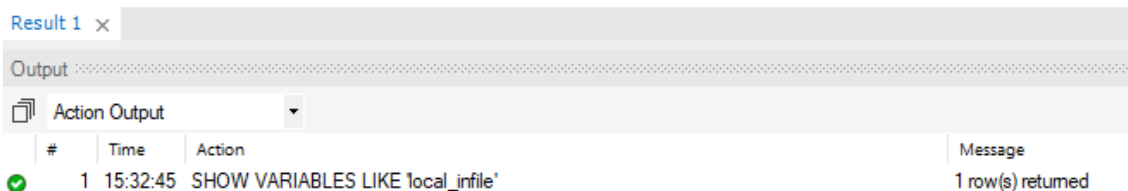
Posteriormente se crean las tablas y se cargaran con los datos facilitados en los archivos csv. Primero se crean las tablas de dimensión y por último la tabla de hechos, con las restricciones Foreign Key para crear integridad de los datos desde el comienzo.

En el momento de carga de datos hay que tener en cuenta si se permite la subida de archivos desde carpeta locales.

```
139 • SHOW VARIABLES LIKE 'local_infile'; -- No permite subir archivos desde carpetas local
```



Variable_name	Value
local_infile	OFF



#	Time	Action	Message
1	15:32:45	SHOW VARIABLES LIKE 'local_infile'	1 row(s) returned

No permite subir archivos desde carpetas locales. Así que, los archivos se deben encontrar en la ruta donde se encuentra el servidor. Para saber en qué ruta poner los archivos y de donde copiar la ruta de los archivos a subir:

142 • `SHOW VARIABLES LIKE 'secure_file_priv';`

Variable_name	Value
secure_file_priv	C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\

Result 2 x

Output

Action Output

#	Time	Action	Message
1	15:32:45	SHOW VARIABLES LIKE 'local_infile'	1 row(s) returned
2	15:39:41	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned

Este equipo > Windows (C:) > ProgramData > MySQL > MySQL Server 8.0 > Uploads

Nombre	Fecha de modificación	Tipo	Tamaño
SPRINT2_ventas_mundiales	11/05/2026 11:35	Carpeta de archivos	

Se crean las tablas y se cargan sus correspondientes datos:

- Tabla y datos de companias:

43 • `CREATE TABLE IF NOT EXISTS companias (`

44 `company_id VARCHAR(10) PRIMARY KEY,`

45 `company_name VARCHAR(255),`

46 `phone VARCHAR (255),`

47 `email VARCHAR (255),`

48 `country VARCHAR (45),`

49 `website VARCHAR (255));`

50 • `LOAD DATA`

51 `INFILE 'C:/ProgramData//MySQL/MySQL Server 8.0/Uploads/SPRINT2_ventas_mundiales/N1.Ex.8__companies.csv'`

52 `INTO TABLE companias`

53 `FIELDS TERMINATED BY ","`

54 `IGNORE 1 ROWS;`

Output

Action Output

#	Time	Action	Message
8	15:46:54	CREATE TABLE IF NOT EXISTS companias (company_id VARCHAR(10) PRIMARY K...	0 row(s) affected
9	15:46:59	LOAD DATA INFILE 'C:/ProgramData//MySQL/MySQL Server 8.0/Uploads/SPRINT2...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Hay que tener en cuenta que las diagonales se tienen que modificar en la ruta copiada para que no leas combinaciones. Se puede cambiar la dirección de la diagonal (de derecha a izquierda /) o añadir una diagonal más a la que viene por defecto (doble diagonal de izquierda a derecha \).

Se revisa la tabla subida:

```

209 -- revision tabla:
210 • SELECT *
211 FROM companias;

```

Result Grid						
	company_id	company_name	phone	email	country	website
▶	b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
	b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
	b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
	b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
	b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netfix.com/settings
	b-2242	Donec Ltd	01 25 51 37 37	at.iaculis@hotmail.couk	Norway	https://nytimes.com/user/110
	b-2246	Sed Nunc Ltd	02 62 64 73 48	nibh@yahoo.org	United Kingdom	https://cnn.com/one
	b-2250	Sed Nunc Ltd	02 62 64 73 48	nibh@yahoo.org	United Kingdom	https://cnn.com/one

companias 4 x

Output

#	Time	Action	Message
✓	17 11:45:08	LOAD DATA INFILE 'C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\SPRINT2_ve...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
✓	18 11:50:08	SELECT * FROM companias	100 row(s) returned

- Tabla y datos de tarjetas_credito:

```

159 • CREATE TABLE IF NOT EXISTS tarjetas_credito (
160     id VARCHAR(45) PRIMARY KEY,
161     user_id VARCHAR(45),
162     iban VARCHAR (250),
163     pan VARCHAR (250),
164     pin VARCHAR(45),
165     cvv VARCHAR(45),
166     track1 VARCHAR (250),
167     track2 VARCHAR (250),
168     expiring_date DATE);
169 • LOAD DATA INFILE "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\SPRINT2_ventas_mundiales\W1.Ex.8__ credit_cards.csv"
170 INTO TABLE tarjetas_credito
171 FIELDS TERMINATED BY ","
172 IGNORE 1 ROWS
173 (id, user_id, iban, pan, pin, cvv, track1, track2, @expiring_date) -- Se ve en csv que el formato de la fecha no coincide con
174 SET expiring_date = STR_TO_DATE(@expiring_date, '%m/%d/%y'); -- Se modifica el formato del campo, para ello hemos de crear v
175

```

Action Output			
#	Time	Action	Message
✓	6 16:32:37	CREATE TABLE IF NOT EXISTS tarjetas_credito (id VARCHAR(45) PRIMARY KEY, ...	0 row(s) affected
✓	7 16:32:40	LOAD DATA INFILE "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\SPRI...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

Como el formato de las fechas en el campo expiring_date no coincide con el tipo de dato DATE, en el momento de subir los archivos a la tabla se cambia el formato el coincidente con el tipo con el que se ha creado el campo. Para ello se han de declarar las columnas, creando una variable ficticia que permita asignar columna al campo.

Comprobación datos en tabla tarjetas_credito:

```

10 • SELECT *
11 FROM tarjetas_credito;

```

	id	user_id	iban	pan	pin	cvv	track1	track2
▶	Cc-4857	276	XX4857591835292505850771	2314242385113924	1819	467	%B2314242385113924^LWCBUDLWCBUD^22...	%B2314242385113924=241010
	Cc-4858	277	XX8581768137002436094025	6582720299715533	3964	817	%B6582720299715533^TIQMVTIQMVI^2404...	%B6582720299715533=241110
	Cc-4859	278	XX7826930491423553609370	8861684536289642	4983	277	%B8861684536289642^COFBGD COFBGD^280...	%B8861684536289642=250210
	Cc-4860	279	XX5559590368835304645299	2481155515498459	6876	661	%B2481155515498459^TIUJTUTIUJTU^31040...	%B2481155515498459=260210
	Cc-4861	280	XX2035182877195191627307	1308930301149557	5710	398	%B1308930301149557^HPOBNZHPBNZ^330...	%B1308930301149557=280510
	Cc-4862	281	XX4774721462463645409758	6715617009807829	4042	174	%B6715617009807829^LDMWTDLDMWTD^33...	%B6715617009807829=221010
	Cc-4863	282	XX1476829664245046207111	3140879819451394	5969	449	%B3140879819451394^OXXJODXXJOD^230...	%B3140879819451394=321010
	Cc-4864	283	XX8380298893385731196159	5793672133649114	8481	139	%B5793672133649114^NHWBYRNHWBYR^30...	%B5793672133649114=230610

tarjetas_credito 3 x

Output

#	Time	Action	Message
7	16:32:40	LOAD DATA INFILE "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\SPRI...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0
8	16:33:35	SELECT * FROM tarjetas_credito	5000 row(s) returned

- Tabla y datos de usuarios_europeos:

```

176 • CREATE TABLE IF NOT EXISTS usuarios_europeos (
177     id VARCHAR(45) PRIMARY KEY,
178     name VARCHAR(45),
179     surname VARCHAR(45),
180     phone VARCHAR(45),
181     email VARCHAR(45),
182     birth_date DATE,
183     country VARCHAR(45),
184     city VARCHAR(45),
185     postal_code VARCHAR(45),
186     address VARCHAR(200),
187     continente VARCHAR(10)); -- Se añade campo porque posteriormente se unificará con tabla usuarios_americanos, para distinguir
188 • LOAD DATA
189 INFILE "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\SPRINT2_ventas_mundiales\W1.Ex.8_european_users.csv"
190 INTO TABLE usuarios_europeos
191 FIELDS TERMINATED BY ","
192 ENCLOSED BY '"' -- Así mismo se observa en csv que ciertos valores están entrecomillados, se añade comando para que los lea la
193 IGNORE 1 ROWS
194 (id, name, surname, phone, email, @birth_date, country, city, postal_code, address)
195 SET birth_date = STR_TO_DATE(@birth_date, '%b %e, %Y'), -- Cambio formato fechas.
196 continente = "europa"; -- Añade información al campo creado

```

#	Time	Action	Message
20	16:22:42	CREATE TABLE IF NOT EXISTS usuarios_europeos (id VARCHAR(45) PRIMARY KEY...	0 row(s) affected
21	16:22:45	LOAD DATA INFILE "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\SPRI...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0

Al cargar los datos se tienen en cuenta que hay campos acotados por dobles comillas (""), los valores están separados por comas y se cambia el formato de los valores del campo birth_date para que coincidan con el tipo de dato DATE. Se crea también un nuevo campo, donde se añadirá el origen de la tabla, pues más adelante se unificará con la tabla usuarios_americanos.

- Tabla y datos de usuarios_americanos:

```

198 • CREATE TABLE IF NOT EXISTS usuarios_americanos (
199     id VARCHAR(45) PRIMARY KEY,
200     name VARCHAR(45),
201     surname VARCHAR(45),
202     phone VARCHAR(45),
203     email VARCHAR(45),
204     birth_date DATE,
205     country VARCHAR(45),
206     city VARCHAR(45),
207     postal_code VARCHAR(45),
208     address VARCHAR(200),
209     continente VARCHAR(15)); -- Se añade campo porque posteriormente se unificará con tabla usuarios_americanos, para disti
210 • LOAD DATA
211     INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\SPRINT2_ventas_mundiales\\W1-Ex.8__ american_users.csv"
212     INTO TABLE usuarios_americanos
213     FIELDS TERMINATED BY ","
214     ENCLOSED BY '"' -- Asi mismo se observa en csv que ciertos valores están entrecomillados, se añade comando para que los le
215     IGNORE 1 ROWS
216     (id, name, surname, phone, email, @birth_date, country, city, postal_code, address)
217     SET birth_date = STR_TO_DATE(@birth_date, '%b %e, %Y'), -- Cambio formato fechas.
218     continente = "norte_america"; -- Añade informacion al campo creado para diferenciarla de la usuarios_europeos

```

Output				
Action Output				
#	Time	Action	Message	
✓ 18	16:45:09	CREATE TABLE IF NOT EXISTS usuarios_americanos (id VARCHAR(45) PRIMARY K...	0 row(s) affected	
✓ 19	16:45:38	LOAD DATA INFILE "C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\SPRI...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	

Se han aplicado los mismos cambios que en la tabla usuarios_europeos.

Se realiza union de ambas tablas para crear una tabla de usuarios_globales:

```

220 -- Nueva tabla que une usuarios_americanos y usuarios_europeos
221 • CREATE TABLE usuarios_globales AS
222     SELECT *
223     FROM usuarios_europeos
224     UNION ALL
225     SELECT *
226     FROM usuarios_americanos;

```

Output				
Action Output				
#	Time	Action	Message	
✓ 11	16:36:08	SELECT * FROM usuarios_americanos	3990 row(s) returned	
✓ 12	16:37:53	CREATE TABLE usuarios_globales AS SELECT * FROM usuarios_europeos UNION AL...	7980 row(s) affected Records: 7980 Duplicates: 0 Warnings: 0	

Se reestablece Primary Key sobre el campo id en la tabla usuarios globales, pues al realizar la unión se ha perdido.

```

228 • ALTER TABLE usuarios_globales
229     ADD PRIMARY KEY (id);

```

Output				
Action Output				
#	Time	Action	Message	
✓ 21	16:46:53	CREATE TABLE usuarios_globales AS SELECT * FROM usuarios_europeos UNION AL...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings:	
✓ 22	16:47:10	ALTER TABLE usuarios_globales ADD PRIMARY KEY (id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	

Comprobación datos en tabla usuarios_globales:

```
231 • SELECT *
232 FROM usuarios_globales;
233
```

	id	name	surname	phone	email	birth_date	country	city	postal_code	address	continente
▶	1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	1985-11-17	United States	New York	10001	348-7818 Sagittis St.	norte_america
	10	Robert	McCarthy	(324) 746-6771	fermentum@protonmail.com	1984-04-30	United States	San Jose	95101	P.O. Box 773	norte_america
	100	Melodie	Mclean	1-677-221-7152	risus.varius@google.ca	1989-09-15	United States	San Jose	95101	Ap #644-8492 Sagittis St.	norte_america
	1000	Amkjrjv	Qbulrxbp	+48-258-9936	amkjrjv.qbulrxbp@example.com	1970-05-17	Germany	Stuttgart	70173	215 Qbulrxbp St	europa
	1001	Nfrlrb	Oydaibwg	+94-121-2522	nfrlrb.oydaibwg@example.com	1994-03-04	Germany	Cologne	50667	121 Oydaibwg St	europa
	1002	Ijbfmd	Jbddzhvp	+70-120-3668	ijbfmd.jbddzhvp@example.com	2001-09-27	Germany	Munich	80331	412 Jbddzhvp St	europa
	1003	Uycig	Sfbdymzj	+58-123-6968	uycig.sfbdymzj@example.com	1981-01-20	Germany	Stuttgart	70173	735 Sfbdymzj St	europa
	1004	Yjqurq	Ojizvgqi	+77-944-2340	yjqurq.ojizvgqi@example.com	1954-07-27	Germany	Munich	80331	685 Ojizvgqi St	europa

usuarios_globales 4 x

Output

#	Time	Action	Message
✓ 22	16:47:10	ALTER TABLE usuarios_globales ADD PRIMARY KEY (id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
✓ 23	16:48:56	SELECT * FROM usuarios_globales	5000 row(s) returned

Se borran las tablas usuarios_europeos y usuarios_americanos, para no duplicar información. En el caso de que se tengan que añadir, modificar o eliminar registros sobre estas tablas se realizarán los cambios sobre la tabla usuarios_globales.

```
233 -- Eliminacion tablas usuarios_europeos y usuarios_americanos:
234 • DROP TABLES usuarios_americanos, usuarios_europeos;
235
```

Output

#	Time	Action	Message
✓ 23	16:48:56	SELECT * FROM usuarios_globales	5000 row(s) returned
✓ 24	16:53:56	DROP TABLES usuarios_americanos, usuarios_europeos	0 row(s) affected

- Tabla y datos de transacciones:

```
236 • CREATE TABLE IF NOT EXISTS transacciones (
237     id VARCHAR (255) PRIMARY KEY,
238     card_id VARCHAR (10),
239     business_id VARCHAR (10),
240     timestamp TIMESTAMP,
241     amount DECIMAL(10, 2),
242     declined BOOLEAN,
243     product_ids VARCHAR (50),
244     user_id VARCHAR (10),
245     lat FLOAT,
246     longitud FLOAT,
247     CONSTRAINT comp_fk FOREIGN KEY (business_id) REFERENCES companias(company_id), -- Se añaden las restricciones
248     CONSTRAINT tj_cred_fk FOREIGN KEY (card_id) REFERENCES tarjetas_credito(id), -- para crear las relaciones ent
249     CONSTRAINT us_glb_fk FOREIGN KEY (user_id) REFERENCES usuarios_globales(id));
250 • LOAD DATA
251 INFILE "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\SPRINT2_ventas_mundiales\N1.Ex.6__ transactions.csv"
252 INTO TABLE transacciones
253 FIELDS TERMINATED BY ";" -- Abriendo csv en bloc de notas (documento plano), los valores no estan separados por co
254 IGNORE 1 ROWS;
255
```

Output

#	Time	Action	Message
✓ 27	17:01:44	CREATE TABLE IF NOT EXISTS transacciones (id VARCHAR (255) PRIMARY KEY, ...	0 row(s) affected
✗ 28	17:01:48	LOAD DATA INFILE "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\SPRI...	Error Code: 2013. Lost connection to MySQL server during query

Se crean las restricciones FK para los campos card_id, business_id y user_id, para crear las relaciones entre las tablas que conforman este modelo.

Comprobamos datos en tabla transacciones:

255 • SELECT *

256 FROM transacciones;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

Fetch rows

id	card_id	business_id	timestamp	amount	declined	product_ids
00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	2024-08-28 07:16:46	395.43	0	16, 26, 97, 87
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	155.63	0	66, 69, 87
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2020-07-14 15:37:45	326.01	0	30, 11, 16, 81
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2017-09-04 19:44:53	161.60	0	72
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	2017-01-05 18:19:25	148.91	0	18
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	2023-09-23 04:51:43	294.59	0	35, 33, 19
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	2023-12-31 00:06:36	383.63	0	93, 55, 28, 91
0009D494-6245-4DF9-955D-2C084191CFFB	CcS-8483	b-2526	2017-07-18 07:52:02	197.80	0	55, 8, 72

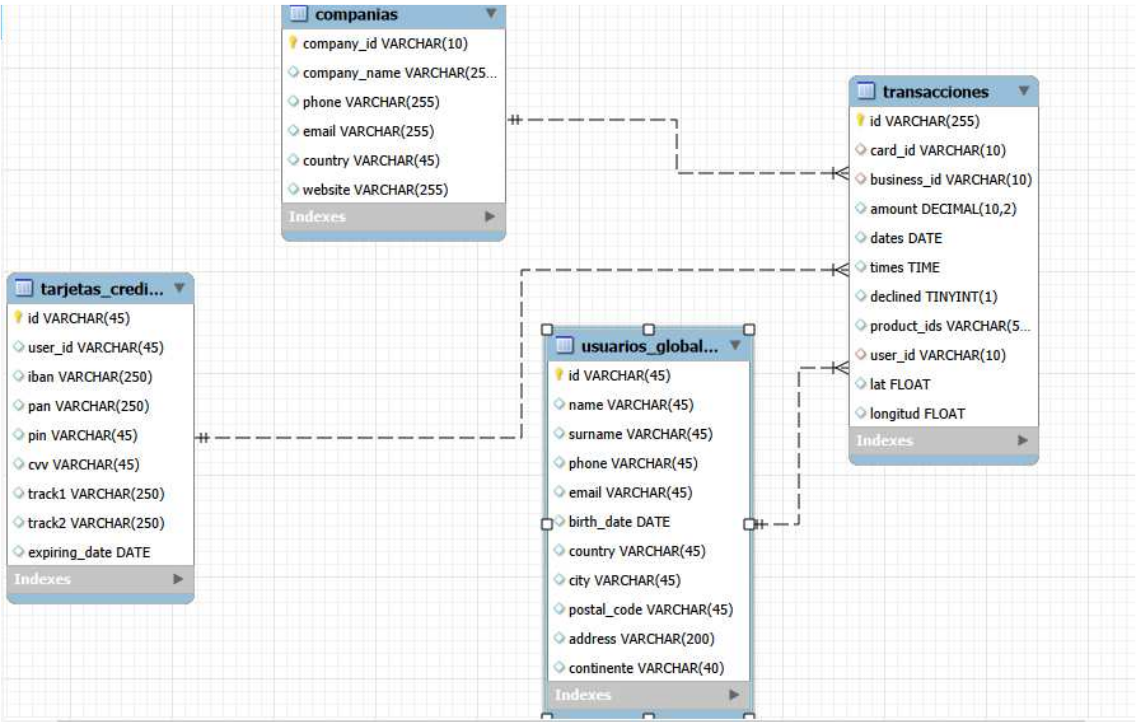
transacciones 7 x

Output

Action Output

#	Time	Action	Message
✓ 1	17:08:27	SELECT * FROM transacciones	100000 row(s) returned

El modelo en estrella de la base de datos ventas mundiales tiene el siguiente diagrama.



Ejercicio 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```
259 #Ejercicio 9
260 -- Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.
261 • SELECT usuarios_globales.id id, usuarios_globales.name nombre, n_transacciones.total_transacciones total_transacciones
262 FROM usuarios_globales
263 JOIN (SELECT transacciones.user_id, count(transacciones.id) as total_transacciones
264 FROM transacciones
265 WHERE declined = 0
266 GROUP BY transacciones.user_id
267 HAVING total_transacciones > 80) AS n_transacciones
268 ON usuarios_globales.id = n_transacciones.user_id
269 GROUP BY usuarios_globales.id;
270
```

Result Grid

id	nombre	total_transacciones
185	Molly	107
289	Dxwgi	91
318	Bnyr	86

Result 8 x

Output

Action Output

#	Time	Action	Message
1	17:08:27	SELECT * FROM transacciones	100000 row(s) returned
2	17:13:35	SELECT usuarios_globales.id id, usuarios_globales.name nombre, n_transacciones.total_t...	3 row(s) returned

Se muestran los usuarios que han realizado más de 80 transacciones donde el pago ha sido aceptado.

Ejercicio 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules

```
1 #Ejercicio 10
2 -- Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec l
3 • SELECT tarjetas_credito.iban iban, ROUND(AVG(transacciones.amount),2) AS media_importe
4 FROM tarjetas_credito
5 JOIN transacciones
6 ON tarjetas_credito.id = transacciones.card_id
7 JOIN companias
8 ON transacciones.business_id = companias.company_id
9 WHERE companias.company_name = "Donec Ltd" and declined = 0
10 GROUP BY tarjetas_credito.iban;
11
12 # Nivel 2
13
```

Result Grid

iban	media_importe
XX911406401125586307586805	356.25
SK9446370242474562577506	142.96
XX776752917845952975555640	257.37
XX413827362289719304908990	139.59
XX347787246070769610780308	240.41

Result 13 x

Output

Action Output

#	Time	Action	Message
6	17:17:56	SELECT tarjetas_credito.iban iban, ROUND(AVG(transacciones.amount),2) AS media_i...	370 row(s) returned
7	17:18:27	SELECT tarjetas_credito.iban iban, ROUND(AVG(transacciones.amount),2) AS media_i...	370 row(s) returned

NIVEL 2

Ejercicio 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

```
286 -- Mostra la data de cada transacció juntament amb el total de les vendes.
287 -- Se utiliza dos CTE (subconsultas) y una funcion ventana
288 WITH sumas_ventas AS (SELECT timestamp dates, sum(amount) AS total_ventas -- Primera subconsulta: Seleccio
289                        FROM transacciones
290                        WHERE declined = "0" -- Marca las transacciones que NO han sido rechazadas.
291                        GROUP BY dates),
292 ranking AS (SELECT dates, total_ventas, DENSE_RANK() OVER(ORDER BY total_ventas DESC) AS prueba -- Seg
293             FROM sumas_ventas) -- Esta funcion permite crear un ranking donde que tiene en cuenta las
294 SELECT dates, total_ventas
295 FROM ranking -- Se muestra la consulta solicitada.
296 WHERE prueba <= 5 ; -- Solicitando los 5 primeros días con mayores ventas, si hubiera más de un día con el
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

dates	total_ventas
2021-04-06 14:25:26	876.66
2021-12-19 02:11:08	858.60
2021-12-19 01:52:10	858.60
2016-06-07 19:37:51	858.55
2018-06-22 01:15:43	845.91

Result 14 x

Output

Action Output

#	Time	Action	Message
7	17:18:27	SELECT tarjetas_credito.iban iban, ROUND(AVG(transacciones.amount),2) AS media_i...	370 row(s) returned
8	17:19:04	WITH sumas_ventas AS (SELECT timestamp dates, sum(amount) AS total_ventas -- Pri...	6 row(s) returned

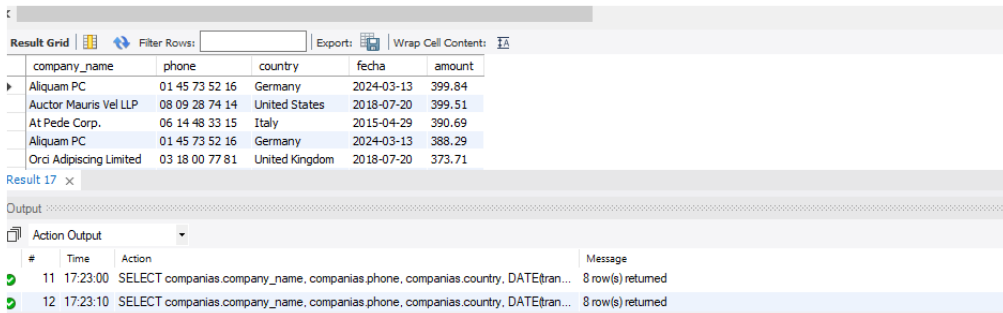
Con tal de mostrar los días que ocupan las 5 primeras posiciones en días con mayores ventas, respetando si hay empates realizo una función ventana `DENSE_RANK()` para enumerar las posiciones, permitiendo repetición, de la subconsulta llamada `sumas_ventas` en la que se realiza una función de agregación por día junto con el total de las ventas cobradas ese día.

Posteriormente se llama a la CTE `ranking` para que muestre los días que tengan un numero en el ranking llamado `prueba` inferior o igual a cinco.

Ejercicio 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

```
302 • SELECT companias.company_name, companias.phone, companias.country, DATE(transacciones.timestamp) fecha, transacciones.amount
303 FROM companias
304 JOIN transacciones
305 ON companias.company_id = transacciones.business_id
306 WHERE (transacciones.amount BETWEEN 350 AND 400)
307 AND (DATE(transacciones.timestamp) in ("2015-04-29", "2018-07-20", "2024-03-13")) -- restricciones solicitadas
308 AND declined = 0
309 ORDER BY transacciones.amount DESC;
```

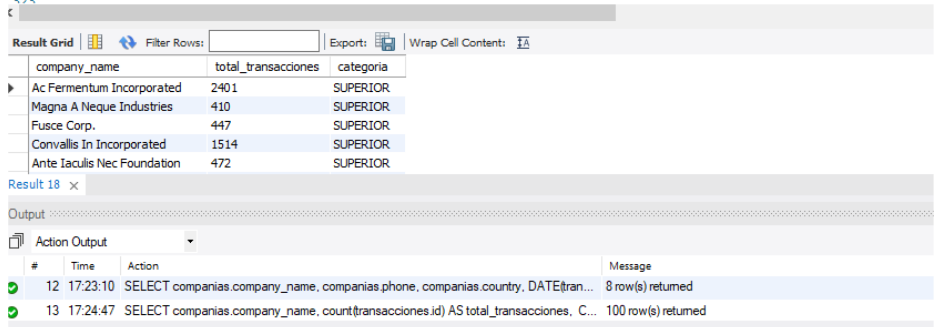


Se unen las tablas compañías y transacciones con filtros solicitados, añadiendo el hecho de que la venta a tenido que ser cobrada.

Ejercicio 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen igual o més de 400 transaccions o menys.

```
316 • SELECT companias.company_name, count(transacciones.id) AS total_transacciones,
317 CASE -- se crea un nuevo campo para la consulta en el que se asigna un valor dependiendo de una condición
318 WHEN count(transacciones.id) >=400 THEN 'SUPERIOR' ELSE 'INFERIOR' END AS categoria -- condición
319 FROM companias
320 JOIN transacciones
321 ON companias.company_id = transacciones.business_id
322 GROUP BY companias.company_name;
```



Se crea nuevo campo solicitado con la estructura condicional marcando los parámetros solicitados

Ejercicio 4

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

Se comprueba existencia del registro:

```
6 -- Primero se mira si existe registro a eliminar:
7 • SELECT *
8 FROM transacciones
9 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
10 -- Se elimina registro
```

sult Grid							
Filter Rows:							
Edit:							
Export/Import:							
Wrap Cell Content:							
id	card_id	business_id	timestamp	amount	declined	product_ids	use
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	155.63	0	66, 69, 87	438
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transacciones 20 x			
put			
Action Output			
#	Time	Action	Message
1	17:27:35	SELECT * FROM transacciones WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1C...	1 row(s) returned

Se elimina registro de la tabla transacciones. Para ello se ha de desactivar el modo de seguridad, reactivándolo después del cambio.

```
-- Se elimina registro.
• SET SQL_SAFE_UPDATES = 0; -- Se desactiva el update_safe_mode
• DELETE FROM transacciones
  WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
• SET SQL_SAFE_UPDATES = 1;
```

Action Output			
#	Time	Action	Message
2	17:29:09	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
3	17:29:11	DELETE FROM transacciones WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1C...	1 row(s) affected
4	17:29:14	SET SQL_SAFE_UPDATES = 1	0 row(s) affected

Se comprueba eliminación:

```
335 -- Se comprueba si registro sigue apareciendo en la tabla:
336 • SELECT *
337 FROM transacciones
338 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
```

Result Grid | | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitud
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

ransacciones 21 x

Output

Action Output

#	Time	Action	Message
4	17:29:14	SET SQL_SAFE_UPDATES = 1	0 row(s) affected
5	17:30:02	SELECT * FROM transacciones WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1..."	0 row(s) returned

Ejercicio 5

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

```
44 • CREATE VIEW VistaMarketing AS -- Se crea la vista. Esta vista se guarda en la BBDD, se podra visualizar/llamar siempre se
45 SELECT companias.company_name, companias.phone, companias.country, ROUND(AVG(transacciones.amount), 2) AS media_compras
46 FROM companias
47 JOIN transacciones
48 ON companias.company_id = transacciones.business_id
49 WHERE declined = 0
50 GROUP BY companias.company_name, companias.phone, companias.country;
51 • SELECT * FROM VistaMarketing; -- Se visualiza la vista.
52
```

The screenshot shows a database interface with a command window and a results grid. The command window displays the SQL code for creating and querying the 'VistaMarketing' view. The results grid shows the output of the 'SELECT * FROM VistaMarketing' query, displaying a table with four columns: company_name, phone, country, and media_compras. The table contains five rows of data, sorted by media_compras in descending order. Below the results grid, there is an 'Action Output' section showing the execution of the SQL commands, including the creation of the view and the execution of the query, with the number of rows affected and returned.

company_name	phone	country	media_compras
Eget Tincidunt Dui Institute	05 35 93 32 44	Netherlands	254.62
Neque Tellus Imperdiet Corp.	09 15 42 22 11	Ireland	263.86
Fusce Corp.	08 14 97 58 85	United States	265.12
Mus Aenean Eget Foundation	06 25 15 52 43	Sweden	256.00
Aliquam Taculis Lacus Corp.	04 43 07 91 26	Belgium	260.20

Output:

#	Time	Action	Message
6	17:32:47	CREATE VIEW VistaMarketing AS -- Se crea la vista. Esta vista se guarda en la BBDD, se podra visualizar/llamar siempre se	0 row(s) affected
7	17:32:52	SELECT * FROM VistaMarketing	100 row(s) returned

Se crea una vista con la consulta solicitada. Esta vista se guarda para poder ser utilizada cada vez que se solicite. Es como una tabla ficticia, que tendrá solo los campos con los que fue creada y solo se podrán solicitar visualizar estos con el comando simple SELECT.

NIVEL 3

Ejercicio 1

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon: Quantes targetes estan actives?

```
0 • CREATE TABLE estado_tarjetas AS -- Se crea nueva tabla
1 WITH operaciones_tarjetas AS (SELECT card_id, declined, timestamp dates, ROW_NUMBER()
2                               OVER ( PARTITION BY card_id ORDER BY timestamp DESC) AS indice_transaccior
3                               FROM transacciones), -- En esta CTE con una funcion ventana creo subgrupos por card_id en las
4 seleccion_fechas AS (SELECT card_id, dates, declined
5                       FROM operaciones_tarjetas -- En esta segunda CTE, de los subgrupos creados, filtro por las tres ul
6                       WHERE indice_transacciones <=3),
7 total_declined AS (SELECT card_id, SUM(declined) AS p_estado
8                   FROM seleccion_fechas
9                   GROUP BY card_id) -- agrego los registros de las trarjetas para sumar el valor declined. Si suma 3, tc
0 SELECT card_id, CASE -- con un condicional segun la suma calculada registro estado de la tarjeta
1                   WHEN p_estado = 3 THEN 'Inactiva' ELSE 'Activa' END AS estado
2 FROM total_declined;
3
4 • SELECT count(estado)
5 FROM estado_tarjetas
6 WHERE estado = "Activa";
7
8 -- RESPUESTA: Hay 4995 tarjetas activas. Es decir que han registrado cobro en una o más de las últimas tres transacciones real
```

Action Output		
#	Time	Action
8	17:35:56	CREATE TABLE estado_tarjetas AS -- Se crea nueva tabla WITH operaciones_tarjetas... Error Code: 1054. Unknown column 'dates' in 'window order by'
9	17:36:13	CREATE TABLE estado_tarjetas AS -- Se crea nueva tabla WITH operaciones_tarjetas... 5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
10	17:36:29	SELECT count(estado) FROM estado_tarjetas WHERE estado = "Activa" 1 row(s) returned

Primero creo una CTE operación_tarjetas, que con una función ventana agrupe en distintos según los días por venta (partición) y después enumere (row_number) cada día de más reciente a más antiguo, reiniciando la enumeración en cada grupo (partición).

Despues, en la CTE selección_fechas, filtro cada grupo por los tres últimos días.

Por último, en la CTE total_declined, agrupo por tarjeta y sumo los valores asociados a ellas por cada operación. Si el valor es 3 significa que esa tarjeta ha sido denegada en las 3 ultimas operaciones, de lo contrario, en al menos una operación, la tarjeta ha sido aceptada. Por lo que se crea un campo donde ponga una cosa u otra según cumpla condición.

Posteriormente visualizo tabla para ver cuantas tarjetas están activas.